

# JAVASCRIPT

## Documentation

---

## Table of Contents

---

1. Introduction to JavaScript
2. Basics of JavaScript
3. Control Flow
4. Functions
5. Arrays and Objects
6. DOM Manipulation
7. Event Handling
8. Callback Functions
9. Promises
10. Async and Await
11. Closures
12. Summary

# 1. Introduction to JavaScript

---

## 1.1 What is JavaScript?

- Created by Brendan Eich.
- Initially built to add simple interactivity to pages in the Netscape browser.
- Now one of the three core technologies of web development, alongside HTML and CSS.

## 1.2 Where Does JavaScript Run?

- Browser — client-side scripting for interactivity and DOM updates.
- Node.js — server-side scripting for backend applications and APIs.
- Cross-platform — desktop, mobile, and IoT applications using frameworks.

## 1.3 The Web Development Analogy

| Technology | Role                                                       |
|------------|------------------------------------------------------------|
| HTML       | Structure — headings, paragraphs, buttons                  |
| CSS        | Styling — colors, layout, animations                       |
| JavaScript | Behavior — click actions, form validation, dynamic content |

## 2. Basics of JavaScript

### 2.1 Variables

JavaScript provides three keywords for declaring variables, each with different rules:

| Keyword      | Description                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------------------|
| <b>var</b>   | Used in older JavaScript code (pre-2015). Function-scoped and can be redeclared, which often leads to bugs.     |
| <b>let</b>   | Introduced in ES6 (2015). Must be declared before use and cannot be redeclared in the same scope. Block-scoped. |
| <b>const</b> | Introduced in ES6 (2015). Cannot be redeclared or reassigned. Block-scoped — the recommended default.           |

#### When to Use var, let, or const

1. Always declare variables — never rely on implicit globals.
2. Always use const if the value should not change.
3. Always use const if the type should not change (e.g. arrays and objects).
4. Only use let if const will not work for that variable.
5. Only use var if you must support very old browsers.

```
const siteName = "ApexaiQ"; // value never reassigned
let score = 0;               // value will change
score = score + 10;

var legacyFlag = true;       // avoid unless required
```

### 2.2 Data Types

| Type             | Example                       |
|------------------|-------------------------------|
| <b>String</b>    | "ApexaiQ"                     |
| <b>Number</b>    | 25, 3.14                      |
| <b>Boolean</b>   | true, false                   |
| <b>Null</b>      | Intentional empty value       |
| <b>Undefined</b> | Declared but not yet assigned |
| <b>Object</b>    | Collection of key-value pairs |

| Type  | Example                                  |
|-------|------------------------------------------|
| Array | Ordered collection of values of any type |

## 2.3 Operators

| Category   | Operators                                                                                    |
|------------|----------------------------------------------------------------------------------------------|
| Arithmetic | <code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>           |
| Comparison | <code>==</code> , <code>===</code> , <code>!=</code> , <code>&lt;</code> , <code>&gt;</code> |
| Logical    | <code>&amp;&amp;</code> (AND), <code>  </code> (OR), <code>!</code> (NOT)                    |
| Assignment | <code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>       |

## 3. Control Flow

---

### 3.1 Conditional Statements

- **if** — executes a block of code if a specified condition is true.
- **else** — executes a block of code if the same condition is false.
- **else if** — tests a new condition when the first condition is false.
- **switch** — executes one of many alternative blocks of code based on a value.

```
const hour = 14;

if (hour < 12) {
  console.log("Good morning");
} else if (hour < 18) {
  console.log("Good afternoon");
} else {
  console.log("Good evening");
}
```

### 3.2 Loops

- **for** — loops through a block of code a set number of times.
- **for/in** — loops through the properties of an object.
- **for/of** — loops through the values of any iterable (arrays, strings, etc.).
- **while** — loops through a block of code while a condition remains true.
- **do/while** — same as while, but runs the block at least once before checking the condition.

```
const colors = ["red", "green", "blue"];

for (let i = 0; i < colors.length; i++) {
  console.log(colors[i]);
}

for (const color of colors) {
  console.log(color);
}
```

## 4. Functions

---

### 4.1 What is a Function?

- A block of code designed to perform a specific task.
- Runs only when it is called, or invoked.
- Helps in reusing code across a program.

#### Key Points

- Parameters — the input values a function accepts.
- Return — the output value a function produces.
- Scope — Global (accessible everywhere), Local (inside a function), or Block (inside { } with let/const).

### 4.2 Types of Functions

| Type                         | Description                                                                                    |
|------------------------------|------------------------------------------------------------------------------------------------|
| <b>Function Declaration</b>  | Defined with the function keyword. Hoisted, so it can be called before it appears in the code. |
| <b>Function Expression</b>   | A function stored in a variable. Not hoisted — can only be used after it is defined.           |
| <b>Arrow Function (ES6)</b>  | Shorter syntax using =>. Does not have its own this, which matters inside objects and classes. |
| <b>Anonymous Function</b>    | A function without a name, often used as a callback.                                           |
| <b>IIFE</b>                  | Immediately Invoked Function Expression — runs automatically as soon as it is defined.         |
| <b>Higher-Order Function</b> | A function that takes another function as an argument, or returns a function.                  |
| <b>Recursive Function</b>    | A function that calls itself to solve a problem in smaller steps.                              |

```
// Function declaration (hoisted)
function greet(name) {
  return `Hello, ${name}!`;
}

// Arrow function
const add = (a, b) => a + b;
```

```
// IIFE
(function () {
  console.log("Runs immediately");
})();
```

## 5. Arrays and Objects

### 5.1 Arrays

An array is an ordered list of values — a container that can hold multiple items, including numbers, strings, objects, or even functions.

#### Common Array Properties & Methods

| Method / Property                                                     | Purpose                                         |
|-----------------------------------------------------------------------|-------------------------------------------------|
| <code>.length</code>                                                  | Returns the total number of items               |
| <code>.push()</code>                                                  | Adds an item at the end                         |
| <code>.pop()</code>                                                   | Removes the last item                           |
| <code>.shift()</code>                                                 | Removes the first item                          |
| <code>.unshift()</code>                                               | Adds an item at the start                       |
| <code>.map()</code> , <code>.filter()</code> , <code>.reduce()</code> | Used for processing and transforming array data |

```
const numbers = [1, 2, 3, 4, 5];

const doubled = numbers.map(n => n * 2);
const evens = numbers.filter(n => n % 2 === 0);
const total = numbers.reduce((sum, n) => sum + n, 0);
```

### 5.2 Objects

An object is a collection of key-value pairs. Keys (properties) are always strings or symbols, and their values can be anything — a string, number, array, function, or another object.

#### Features Provided by Objects

- Accessing properties
- Adding and updating properties
- Deleting properties
- Methods — functions defined inside objects
- Built-in object methods

```
const user = {
  name: "Asha",
  age: 28,
  greet() {
    return `Hi, I'm ${this.name}`;
  }
};
```

```
    },  
  };  
  
  user.age = 29;      // update  
  delete user.age;    // delete
```

## 6. DOM Manipulation

---

### 6.1 What is the DOM?

- The DOM (Document Object Model) is a programming interface for HTML and XML documents.
- It represents the page structure as a tree of nodes — elements, attributes, and text.
- JavaScript can manipulate the DOM to change page content, style, or structure dynamically.

### 6.2 Why the DOM Matters

- Lets developers change content dynamically (e.g. update text or images).
- Enables interaction with users, such as clicking buttons or filling forms.
- Helps create dynamic websites instead of static ones.

### 6.3 Common DOM Manipulation Methods

- Selecting elements
- Changing content
- Changing style
- Creating new elements

```
const title = document.querySelector("#title");
title.textContent = "Welcome!";
title.style.color = "orange";

const newItem = document.createElement("li");
newItem.textContent = "New task";
document.querySelector("ul").appendChild(newItem);
```

## 7. Event Handling

---

### 7.1 What is an Event?

An event is an action that happens in the browser — for example, a user clicking a button, pressing a key, or moving the mouse. JavaScript can "listen" for these events and respond to them.

### 7.2 Common Types of Events

| Event                      | Triggered When                  |
|----------------------------|---------------------------------|
| <b>onclick</b>             | A user clicks an element        |
| <b>onmouseover</b>         | The mouse moves over an element |
| <b>onkeydown / onkeyup</b> | A key is pressed or released    |
| <b>onsubmit</b>            | A form is submitted             |
| <b>onchange</b>            | The value of an input changes   |

```
const button = document.querySelector("#submit");

button.addEventListener("click", () => {
  console.log("Button was clicked!");
});
```

## 8. Callback Functions

---

### 8.1 What is a Callback?

- A function passed as an argument to another function.
- Allows one function to call another function later.
- Useful for tasks that take time, such as reading files or making API calls.
- Makes code more flexible and reusable.

### 8.2 Why Use Callbacks?

| Benefit        | Explanation                                                     |
|----------------|-----------------------------------------------------------------|
| Reusability    | The callback can be changed without touching the main function. |
| Async Handling | Used to wait for tasks like API requests or file reading.       |
| Control Flow   | Defines what should happen once a task completes.               |

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback("Data loaded");  
  }, 1000);  
}  
  
fetchData((result) => console.log(result));
```

## 9. Promises

---

### 9.1 What is a Promise?

- A special JavaScript object used for handling asynchronous operations.
- Represents a value that may be available now, later, or never.

#### The Three States of a Promise

| State     | Meaning                                     |
|-----------|---------------------------------------------|
| Pending   | Waiting for the result                      |
| Fulfilled | Operation completed successfully (resolved) |
| Rejected  | Operation failed (error)                    |

### 9.2 Why Use Promises?

- Avoids callback hell — no deeply nested callbacks.
- Cleaner async handling with `.then()` and `.catch()`.
- Chainable — multiple async operations can run in sequence.

```
function fetchData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => resolve("Data loaded"), 1000);
  });
}

fetchData()
  .then((result) => console.log(result))
  .catch((error) => console.error(error));
```

## 10. Async and Await

---

### 10.1 What is it?

- Async/Await is syntax built on top of Promises.
- Makes asynchronous code look and behave like synchronous code.
- `async` marks a function as one that will return a Promise.
- `await` pauses the function until the Promise is fulfilled or rejected.

### 10.2 Why Use Async and Await?

- Cleaner code that reads like step-by-step instructions.
- Avoids chaining `.then()` calls and nested promises.
- Easier error handling using `try...catch`.
- Considered the modern best practice for handling async tasks.

```
async function loadData() {  
  try {  
    const result = await fetchData();  
    console.log(result);  
  } catch (error) {  
    console.error("Something went wrong:", error);  
  }  
}  
  
loadData();
```

## 11. Closures

---

### 11.1 What is a Closure?

- A function that remembers variables from its outer scope, even after the outer function has finished executing.
- Created automatically whenever a function is defined inside another function.
- Helps with data hiding, encapsulation, and creating private variables.

### 11.2 Why Use Closures?

| Benefit                | Explanation                                            |
|------------------------|--------------------------------------------------------|
| Data Privacy           | Variables stay hidden and cannot be accessed directly. |
| Encapsulation          | Only specific functions can use the hidden variables.  |
| Stateful Functions     | Useful for counters, caching, and event handlers.      |
| Functional Programming | Forms the base of many advanced JavaScript patterns.   |

```
function createCounter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const counter = createCounter();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

## 12. Summary

---

JavaScript is a versatile, cross-platform language that powers interactivity on the web and beyond. This documentation has covered the language from its fundamentals — variables, data types, and operators — through control flow and functions, into data structures like arrays and objects, and on to browser-facing concepts such as the DOM and event handling. Finally, it introduced the asynchronous programming model through callbacks, promises, and `async/await`, closing with closures, one of the language's more advanced but foundational concepts.

**Key Takeaway**

A solid grasp of these fundamentals — especially scope, functions, and asynchronous patterns — provides the foundation needed to build real-world, interactive, and scalable JavaScript applications.